

Diving into Claude Code's Source Code Leak



ENGINEER'S CODEX

MAR 31, 2026



134



2



21

Sh

Engineer's Codex is a publication about real-world software engineering.

[Subscribe](#)

On March 31, 2026, Anthropic accidentally shipped a `.map` sourcemap file inside a Claude Code npm update. In minutes, this was found and was going viral. The 600k lines of code were mirrored, analyzed, ported to Python and other languages, and uploaded to decentralized servers.

Claude Code is known to be notoriously closed down. Their Agent SDKs provide almost no insight into the internals of Claude Code, and Anthropic themselves do their best to keep the source as closed as possible.

How It Happened

Anthropic accidentally included a `.map` sourcemap file in a Claude Code npm package. Boris Cherny, a Claude Code engineer at Anthropic, [confirmed it was plain developer error](#), not a tooling bug. He [followed up](#) with more detail: “Mistakes happen

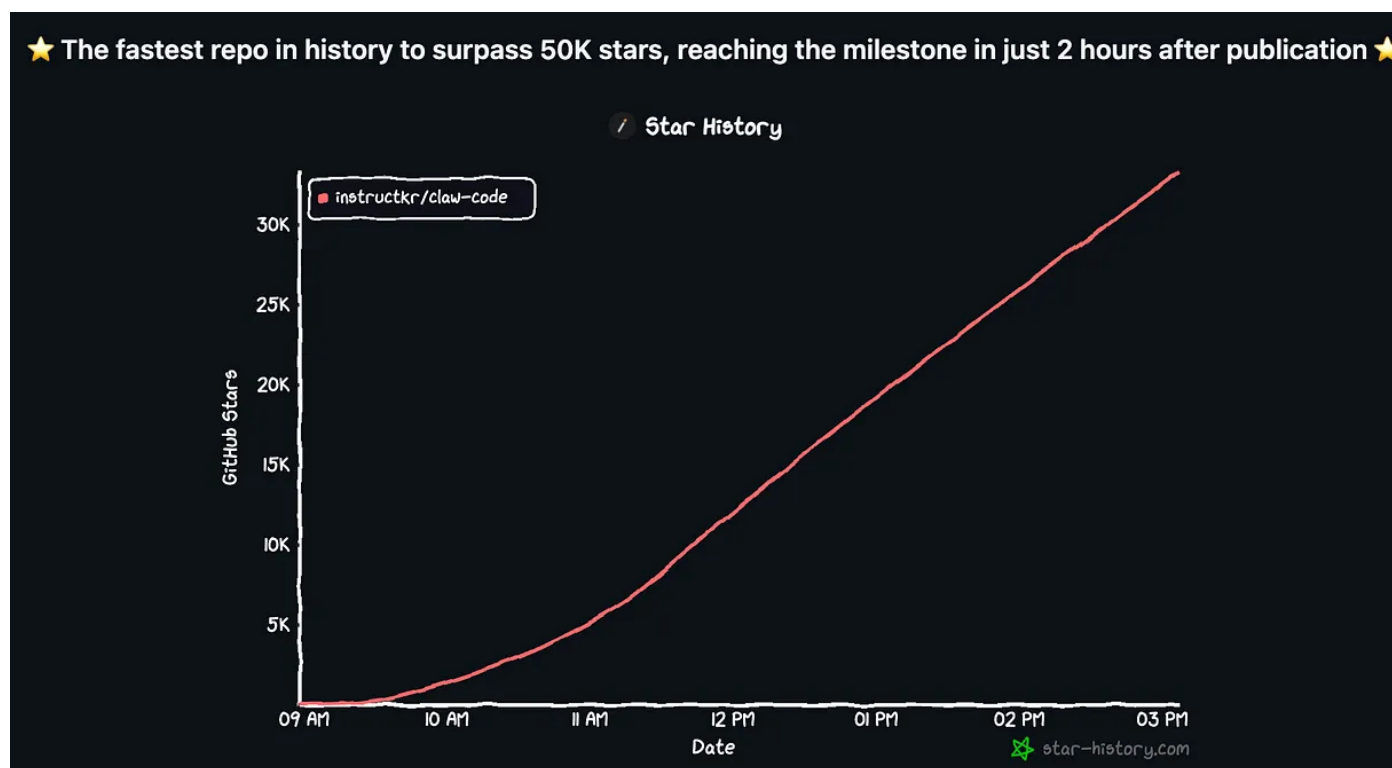
© 2026 Engineer's Codex · [Privacy](#) · [Terms](#) · [Collection notice](#)
[Substack](#) is the home for great culture

[Blameless post-mortems](#), popularized by Google's SRE culture, focus on fixing the system rather than finding someone to fault. The goal is an environment where engineers report mistakes honestly instead of hiding them, which produces better fixes and fewer repeat incidents.

Chaofan Shou ([@Fried_rice](#)) was first to notice and posted a public link to the source. Within minutes the race was on.

Chaos and Legality

The most popular fork was `claw-code` [on GitHub](#), created by @realsigridjin. He cloned the repo, ported it to Python from scratch (using OpenAI's Codex!) to prevent legal issues, and pushed it.



The legal theory is that a clean-room AI rewrite cannot be touched by DMCA. `Claw-code` currently sits at 75,000+ stars and 75,000+ forks at the time of writing.

The legal question nobody has an answer to yet: does a codegen clean-room rebuild violate copyright?

Traditionally: “A clean room build in software engineering is a **methodology used to re-implement existing software or reverse-engineer a system without infringing copyright**. It involves two separate teams: one analyzes the original software to create specifications, while a second "clean" team creates the new product based *only* on those specifications, ensuring no proprietary code is copied.”

This takes months and costs real money. It used to be a meaningful barrier. Now anyone with a Claude Max plan can point an agent at a codebase's tests and have the logic rebuilt overnight. The practice has never been challenged in court.

[Gergely Orosz](#) put it bluntly: even if Anthropic tries to assert copyright, do they win the PR battle of suing an open source project for rebuilding their own AI-written product? And could they even prove it?

Meanwhile, [4nzn](#) uploaded a [stripped version](#) to IPFS with all telemetry removed, security guardrails removed, and all experimental features unlocked. Whether DMCA can reach content stored on IPFS is itself an unresolved legal question.

At the moment, `claw-code` is still up, but non-rewritten forks have been DMCA'd by Anthropic.

Here is an example DMCA request Anthropic sent to a source clone: <https://github.com/github/dmca/blob/master/2026/03/2026-03-31-anthropic.md>

Inside the Source Code

KAIROS: Anthropic's Response to OpenClaw

Hidden behind feature flags called PROACTIVE and KAIROS, the codebase contains

fully built autonomous agent mode that has not been publicly announced.

KAIROS runs in the background, 24/7, without you asking. It's similar to OpenClaw

Every few seconds it receives a heartbeat prompt: "anything worth doing right now
It evaluates what is happening and makes a call: act, or stay quiet. If it acts, it can fix errors, respond to messages, update files, run tasks. Everything Claude Code can already do, just without you initiating it.

It has three exclusive tools that regular Claude Code does not:

- **Push notifications** so it can reach you on your phone or desktop even when the terminal is closed
- **File delivery** so it can send you things it created without being asked
- **Pull request subscriptions** so it can watch your GitHub and react to code changes on its own

It keeps append-only daily logs of everything it noticed, decided, and did. It cannot erase its own history. At night it runs a process the code literally calls `autoDream`, where it consolidates what it learned during the day and reorganizes memory. It persists across sessions. Close your laptop Friday, open it Monday, KAIROS has been working the whole time.

The codebase also contains 44 hidden feature flags and 20+ unshipped features total. Background agents running 24/7, one Claude orchestrating multiple worker Claude agents, cron scheduling, full voice command mode, browser control via Playwright, and agents that sleep and self-resume.

The interesting architectural decision in all of this is the separation of initiative from execution.

Regular Claude Code is reactive: it only acts when you send a message. KAIROS

introduces a proactive loop, which requires a fundamentally different trust model. The agent now needs to decide on its own what is worth doing, which means the quality of judgment becomes far more important than it is in a simple request-response system.

That is a hard problem, and the fact that Anthropic has it fully built and gated suggests they have been working on it for some time.

Anti-Distillation: Actively Poisoning Copycats

Distillation is a process where a smaller, more efficient AI model is trained to mimic the behavior and knowledge of a larger, more powerful “teacher” model, allowing it to perform nearly as well while being much faster and cheaper to run.

Claude Code sends requests with `anti_distillation: ['fake_tools']` flag that inject decoy tool definitions into system prompts. The goal is to poison training data for any competitor trying to distill Claude Code’s behavior into their own model.

There is a second layer via `CONNECTOR_TEXT`: a server-side mechanism that buffers the assistant’s text between tool calls, summarizes it with cryptographic signatures, and returns only those summaries to API traffic recorders. Even if a competitor intercepts API traffic, they only get summaries, not the full reasoning chains.

What makes the fake tools approach interesting is where it operates. It targets the distillation attack surface at data collection time rather than at inference time. If a competitor routes requests through Claude Code to collect training data, they get poisoned tool schemas baked into every prompt, making any model trained on that data less reliable.

The author of [this breakdown](#) estimates a determined team could bypass both mechanisms within an hour using a MITM proxy or the `CLAUDE_CODE_DISABLE_EXPERIMENTAL_BETAS` environment variable. The

stronger deterrent is likely legal rather than technical.

SWE Quiz: Did you know what distillation was?

Distillation is an important concept that comes up in interviews with any company working on their own AI model. Did you know what it was?

SWE Quiz is the best way to figure out what you do and don't know. It's like Anki System Design and AI/ML, with structured courses, quizzes, and case studies.

Model Codenames

The codebase is littered with internal codenames that were never meant to be public.

- **Capybara** (also called Mythos): Anthropic is already on version 8. It has 1M context and a “fast mode.” The code notes it still has issues with over-comment and making false claims.
- **Numbat**: tagged with a comment reading “[MODEL LAUNCH]: Remove this section when we launch numbat”, an upcoming model with an announced launch window baked into the source.
- **Fennec**: speculated by multiple researchers to be Opus 4.6.
- **Tengu**: referenced in the undercover mode, which strips internal codenames from builds going to external repositories.

The undercover mode (`undercover.ts`, 90 lines) has a critical design detail: there is no force-OFF. It is a one-way door. Claude Code operating in external repos cannot reference any of these names, internal Slack channels, or even call itself “Claude Code.”

The reasoning is straightforward from a product security standpoint: you want to

guarantee internal codenames never escape into the wild, so you make suppression automatic and irrevocable rather than opt-in. But a side effect is that Anthropic employees using Claude Code to contribute to open source projects were not disclosing AI authorship in commits, and the tool was built to make sure that stayed the case.

DRM Below the JavaScript Layer

API requests contain placeholder values (`cch=ed1b0`) that Bun's native HTTP stack written in Zig, overwrites with computed hashes before transmission. This cryptographically proves a request originated from a genuine Claude Code binary rather than a third-party tool wrapping the API.

The reason this lives in Zig rather than in JavaScript is deliberate. JavaScript can be patched, monkey-patched, or proxied at runtime. Zig code compiled into the Bun binary cannot be inspected or overridden without recompiling from source. By pushing attestation below the JS layer, Anthropic made it much harder for third-party clients to spoof.

This is likely why tools like OpenCode ran into friction with Anthropic beyond just legal notices: they were being blocked at the API level too. The mechanism is gated behind a compile-time flag, which suggests it may not be active everywhere, but the design intent is clear.

The Memory Architecture

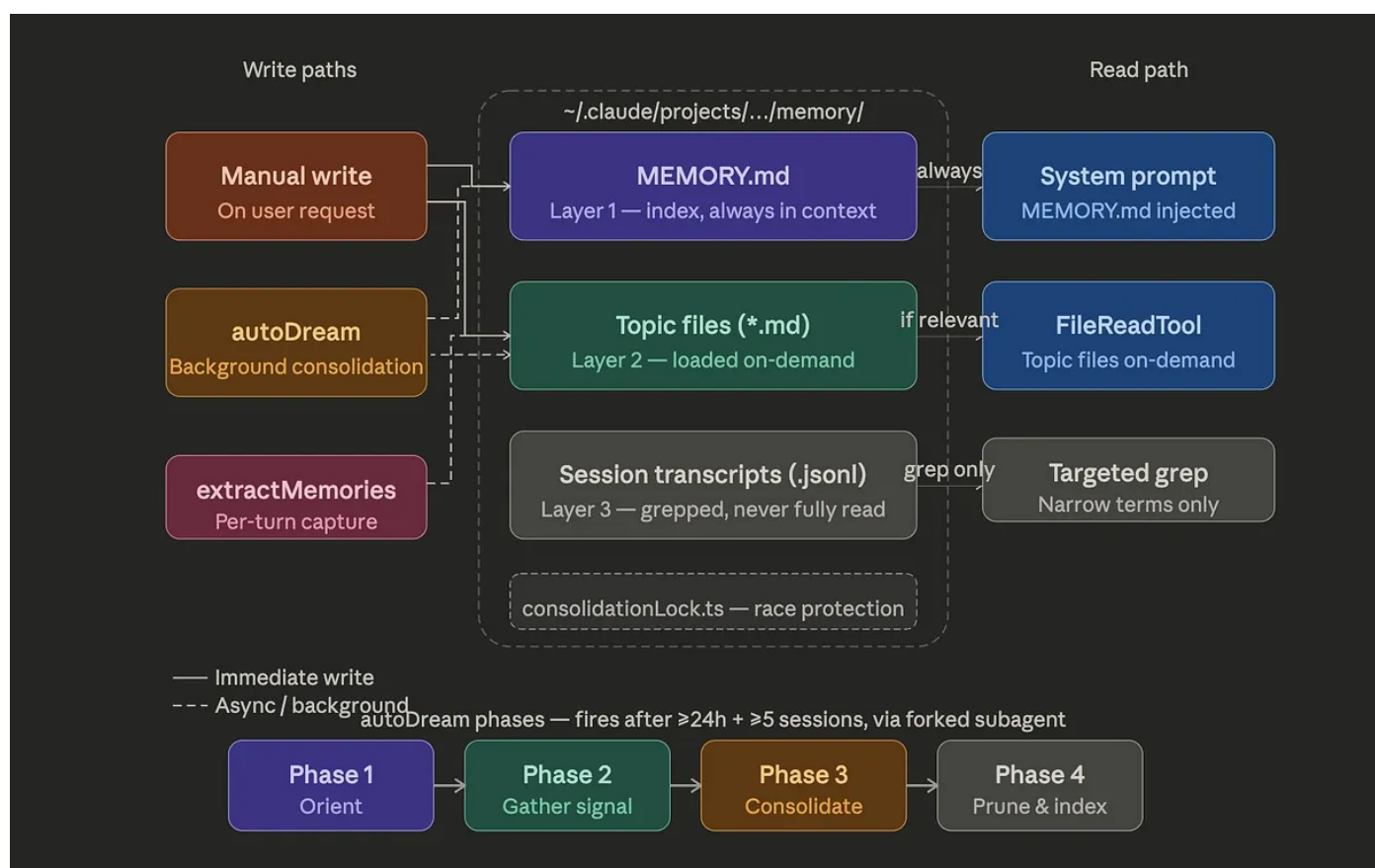
Claude Code's memory system is far more sophisticated than "save a CLAUDE.md file." It operates as a 3-layer index:

1. **Index** (always loaded): just pointers, ~150 characters per line
2. **Topic files** (loaded on demand): actual knowledge
3. **Transcripts** (never read, only grep'd): never loaded into context directly

The write discipline: write to topic file first, then update the index. Never dump content into the index. If a fact can be re-derived directly from the codebase, it is not stored at all.

autoDream consolidates, deduplicates, and removes contradictions from memory in a forked subagent with limited tool access, which prevents it from corrupting the main context while it runs. Memory is treated as a hint, not as truth. The agent verifies before using it.

The core design insight here is bandwidth awareness. Most agent memory implementations load everything into context every turn, which is expensive and introduces noise. Claude Code treats the context window as a scarce resource: the index is cheap and always loaded, topic files are fetched only when relevant, and transcripts are never loaded directly. This keeps the model's working memory clean without losing access to historical information. What they choose not to store matters as much as what they do store.



[Source](#)

Magic Docs: Self-Updating Documentation

[One pattern worth borrowing](#) from the internal codebase: Anthropic employees can create files with a MAGIC DOC header. When an internal build of Claude Code is id it fires off a dedicated subagent that reads the file, updates the documentation for t specified feature, and writes it back. The subagent is restricted to editing that singl file and nothing else.

The result is documentation that stays current automatically, without anyone havin to remember to update it. The tool restriction is the key design choice: scoping the agent to one file prevents it from drifting into unrelated changes while still giving i enough access to do something useful.

Summarizing the Harness

Drop DeepSeek or Gemini into the same Claude Code harness with some optimizat and you may get improved coding ability with those models too.

The harness includes:

- Live repo context loading on every single turn (git branch, recent commits, CLAUDE.md files reread every query)
- Aggressive prompt cache reuse via a stable/dynamic prompt boundary, so Anthropic is not paying full token costs every turn
- Dedicated Grep and Glob tools instead of shell commands, giving the model better-structured search results
- LSP tool access for call hierarchies, symbol definitions, and references
- 5 distinct context compaction strategies, because context overflow is a central engineering problem

- A 25+ event hook system letting you intercept and modify behavior at every stage of execution
- Three subagent execution models (fork, teammate, worktree) with a prompt caching optimization: forked subagents inherit parent context as byte-identical copies, so spawning 5 agents costs barely more than 1

One design decision worth understanding is the prompt cache boundary. Anthropic pays per token, and a long system prompt rebuilt from scratch every turn would be expensive at scale. By splitting the prompt into a stable front half and a dynamic back half via `SYSTEM_PROMPT_DYNAMIC_BOUNDARY`, they make sure the expensive static portions are cached and reused across sessions. Anything tagged `DANGEROUS_uncachedSystemPromptSection` is explicitly marked as cache-breaking, so engineers know the cost before they change it.

Much of this system prompting was embedded directly in the CLI package distributed via npm all along. It was always technically readable. The `.map` file just made it obvious.

The Claude Code team has talked about [Prompt Caching in Claude Code before](#).

Easter Eggs and Human Touches

- `buddy/companion.ts` is an April Fools feature that generates a deterministic creature per user: 18 species, rarity tiers, 1% shiny rate, RPG stats deliberately encoded to evade grep checks
- There are exactly 187 spinner verbs (Source: [Wes Bos](#))

```
15 // Spinner verbs for loading messages
16 export const SPINNER_VERBS = [ 'Accomplishing', 'Actioning', 'Actualizing', 'Architecting', 'Baking', 'Beaming', 'Beboppin',
  'Befuddling', 'Billowing', 'Blanching', 'Blovitating', 'Boogieing', 'Boondoggling', 'Booping', 'Bootstrapping', 'Brewing',
  'Bunning', 'Burrowing', 'Calculating', 'Canoodling', 'Caramelizing', 'Cascading', 'Catapulting', 'Cerebrating', 'Channeling',
  'Channelling', 'Choreographing', 'Churning', 'Clauding', 'Coalescing', 'Cogitating', 'Combobulating', 'Composing', 'Computing',
  'Concocting', 'Considering', 'Contemplating', 'Cooking', 'Crafting', 'Creating', 'Crunching', 'Crystallizing', 'Cultivating',
  'Deciphering', 'Deliberating', 'Determining', 'Dilly-dallying', 'Discombobulating', 'Doing', 'Doodling', 'Drizzling', 'Ebbing',
  'Effecting', 'Elucidating', 'Embellishing', 'Enchanting', 'Envisioning', 'Evaporating', 'Fermenting', 'Fiddle-faddling',
```

```
'Finagling', 'Flambéing', 'Flibbertigibbeting', 'Flowing', 'Flummoxing', 'Fluttering', 'Forging', 'Forming', 'Frolicking',
'Frosting', 'Gallivanting', 'Gallopig', 'Garnishing', 'Generating', 'Gesticulating', 'Germinating', 'Gitifying', 'Grooving',
'Gusting', 'Harmonizing', 'Hashing', 'Hatching', 'Herding', 'Honking', 'Hullabalooing', 'Hyperspacing', 'Ideating', 'Imagining',
'Improvising', 'Incubating', 'Inferring', 'Infusing', 'Ionizing', 'Jitterbugging', 'Julienning', 'Kneading', 'Leavening',
'Levitating', 'Lollygagging', 'Manifesting', 'Marinating', 'Meandering', 'Metamorphosing', 'Misting', 'Moonwalking', 'Moseying',
'Mulling', 'Mustering', 'Musing', 'Nebulizing', 'Nesting', 'Newspapering', 'Noodling', 'Nucleating', 'Orbiting', 'Orchestrating',
'Osmosing', 'Perambulating', 'Percolating', 'Perusing', 'Philosophising', 'Photosynthesizing', 'Pollinating', 'Pondering',
'Pontificating', 'Pouncing', 'Precipitating', 'Prestidigitating', 'Processing', 'Proofing', 'Propagating', 'Puttering',
'Puzzling', 'Quantumizing', 'Razzle-dazzling', 'Razzmatazzing', 'Recombobulating', 'Reticulating', 'Roosting', 'Ruminating',
'Sautéing', 'Scampering', 'Schlepping', 'Scurrying', 'Seasoning', 'Shenaniganing', 'Shimmying', 'Simmering', 'Skedaddling',
'Sketching', 'Slithering', 'Smooshing', 'Sock-hopping', 'Spelunking', 'Spinning', 'Sprouting', 'Stewing', 'Sublimating',
'Swirling', 'Swooping', 'Symbioting', 'Synthesizing', 'Tempering', 'Thinking', 'Thundering', 'Tinkering', 'Tomfoolering',
'Topsy-turvyng', 'Transfiguring', 'Transmuting', 'Twisting', 'Undulating', 'Unfurling', 'Unravelling', 'Vibing', 'Waddling',
'Wandering', 'Warping', 'Whatchamacalliting', 'Whirlpooling', 'Whirring', 'Whisking', 'Wibbling', 'Working', 'Wrangling',
'Zesting', 'Zigzagging',
```

- `print.ts` is 5,594 lines with a single function that is 3,167 lines long and 12 nesting levels deep
- The code has LLM-oriented comments throughout, written for AI agents work on the codebase rather than human readers

March 2026 Was a Security Disaster

The leak did not happen in isolation. It landed one of the worst months in AI developer security on record.

In the same 30-day window:

- **Axios** (100M weekly npm downloads): maintainer account hijacked, a remote access trojan deployed across macOS, Windows, and Linux. The malware self-destructs after execution and deletes itself from `node_modules`. [Google suspect its from North Korean bad actors.](#)
- **LiteLLM** (97M monthly PyPI installs): backdoored with a three-stage attack: credential harvester sweeping SSH keys, AWS/GCP/Azure credentials, Kubernetes configs, crypto wallets, and every LLM API key; then Kubernetes lateral

movement; then a persistent systemd backdoor. Live for three hours before PyF quarantined it

- **Railway** (2M users, 31% of Fortune 500 as customers): CDN misconfiguration leaked authenticated user data to wrong users for 52 minutes
- **Delve**: YC-backed compliance startup allegedly generating fraudulent SOC 2 reports with identical boilerplate across 494 reports, fabricated board meeting evidence, and audit conclusions written before any client submitted evidence
- **Mercor AI**: alleged LAPSUS\$ breach of 939GB source code and 4TB of total data via their TailScale VPN
- **OpenAI Codex**: command injection via branch names discovered December 2025, patched February 2026, disclosed March 2026. Attackers could steal GitHub authentication tokens via unsanitized branch name parameters
- **GitHub Copilot**: injected promotional ads into 1.5M+ pull requests as hidden HTML comments without developer consent. GitHub VP confirmed it and said it was “the wrong judgement call”

The Claude Code leak is, in some ways, the least technically dangerous item on that list. I'll probably cover AI security next!

[Subscribe](#)

What Engineers Building Agents Should Take Away

The source code itself will probably get taken down, reposted, and litigated.

Some things stand out as directly applicable:

Split your system prompt at a stable boundary: The

`SYSTEM_PROMPT_DYNAMIC_BOUNDARY` pattern is a practical response to a real co

problem. Static instructions should never change between sessions. Dynamic content should live after the boundary. Getting this right means your prompt cache actually works and you are not paying to recompute the same tokens on every turn.

- Prompt caching might be obvious to anyone who works with agents daily, but if you don't, know that it's an extremely important concept for keeping costs low.

Magic Docs: Having self-updating documentation is one of those practices that can upgrade any codebase and isn't that hard to do with agents today.

Lots of comments: Generally, too many comments were considered too verbose when humans were the only readers and writers of code. But with agents, comments are useful when detailed, especially when they include decisions + reasoning (basically stuff that's obvious to humans but not to agents).

Permission classification via side-query: Send the command to Claude as a separate query: "Is this command safe?" The model evaluates context, working directory, and user intent. This replaces brittle allowlists with adaptive, context-aware security. Cost is negligible vs. the risk of a false positive.

- This is known as the "critic" pattern.

Subscribe to Engineer's Codex

By Engineer's Codex · Launched 3 years ago

Real-world software engineering explained simply.

By subscribing, you agree Substack's [Terms of Use](#), and acknowledge its [Information Collection Notice](#) and [Privacy Policy](#).



134 Likes • 21 Restacks

← Previous

Discussion about this post

Comments Restacks



Write a comment...



ToxSec 20h

♥ Liked by Engineer's Codex

I was waiting of a few of these. some awesome stuff here.

♥ LIKE (1) 💬 REPLY



Samuel R Holladay 1d

Anthropic engineers rediscover the idea of memory hierarchy and caches when working with context, lmao

♥ LIKE 💬 REPLY



